· Easy. #119 → Pascal's Triangle II

▲ Problem Statement: Given a "rowIndex" (0-indexed), return that particular row of Pascal's Triangle.

Ex → rowIndex = 3 → [1, 3, 3, 1]

▲ Main Problem Solution: We only the required row, so there is no need to store the entire triangle.

· Start with `[1]` & build each row using the previous row.

· To optimize space, we can modify the same vector from right to left, so we only O (rowIndex) extra space.

▲ Brute Force Method: Build the entire Pascal's Triangle up to "rowIndex" & return the required row.

Time :- $O(\text{rowIndex}^2)$
Space :- $O(\text{rowIndex}^2)$

▲ Algorithm: 1| • Create a vector "row" containing "1"

2| • For each row from "1" to "rowIndex" :-

• Add a "1" at the end

• Traverse from right to left.

- Update each element using :-

"row[j] = row[j] + row[j-1]"

3. Return "row"

Important Points → • Start with "[1]"

- Always add "1" at the end of every new row.
- Traverse right to left so previous values aren't overwritten before they're used.
- Only one vector is needed.
- This satisfies the follow-up requirement of O(rowIndex) extra space.

Complexity → Time → $O(rowIndex^2)$
Space → $O(rowIndex^2)$